

Indice- Unidad Colecciones Dinámicas en Java

1	Introducción.....	3
2	Algoritmos y estructuras de datos.....	3
3	Colecciones en Java.....	4
3.1	Interface Collection.....	4
3.2	Interface List.....	5
3.2.1.1	Creación de una nueva lista.....	7
3.2.1.2	Operaciones básicas de la lista: agregar, recuperar, actualizar, eliminar elementos	8
3.2.1.2.1	Adición de elementos a una lista:.....	8
3.2.1.2.2	Recuperar elementos de una lista.....	9
3.2.1.2.3	Eliminación de elementos de una lista.....	9
3.2.1.3	Iteración sobre los elementos de una lista.....	10
3.2.1.4	Búsqueda de un elemento en una lista.....	10
3.2.1.5	Clasificación de una lista.....	11
3.2.1.6	Copiar elementos de una lista en otra.....	11
3.2.1.7	Barajar una lista.....	12
3.2.1.8	Invertir elementos en una lista.....	12
3.2.1.9	Extracción de una porción de una lista.....	12
3.2.1.10	Conversión entre listas y matrices.....	13
3.2.1.11	Video para la práctica de Listas.....	13
3.3	Set.....	13
3.3.1.1	HashSet:.....	14
3.3.1.2	LinkedHashSet:.....	15
3.3.1.3	TreeSet:.....	15
3.3.1.4	Creación de un nuevo conjunto.....	15
3.3.1.5	Realizar operaciones básicas en un conjunto.....	16
3.3.1.5.1	Adición de elementos a un conjunto:.....	16
3.3.1.5.2	Eliminación de un elemento de un conjunto:.....	17
3.3.1.5.3	Compruebe si un conjunto está vacío:.....	17
3.3.1.5.4	Elimine todos los elementos de un conjunto:.....	17
3.3.1.5.5	Obtenga un número total de elementos en un conjunto:.....	17
3.3.1.6	Iteración sobre elementos en un conjunto.....	18
3.3.1.6.1	Uso de un iterador:.....	18
3.3.1.6.2	Uso del bucle for mejorado:.....	18
3.3.1.7	Búsqueda de un elemento en un conjunto.....	18
3.3.1.8	Realización de operaciones masivas entre dos conjuntos.....	19
3.3.1.8.1	Operación de subconjunto:.....	19
3.3.1.8.2	Operación de unión:.....	19
3.3.1.8.3	Operación de intersección:.....	20
3.3.1.8.4	Operación de diferencia:.....	20
3.3.1.9	Conjuntos simultáneos.....	20
3.4	Map.....	21
3.4.1.1	Por qué y cuándo usar mapas:.....	22
3.4.1.2	Implementaciones del mapa.....	22
3.4.1.3	Creación de un nuevo mapa.....	22
3.4.1.3.1	Creación de un HashMap:.....	22
3.4.1.3.2	Creación de un LinkedHashMap:.....	23
3.4.1.3.3	Creación de un TreeMap:.....	23
3.4.1.4	Realización de operaciones básicas en un mapa.....	24
3.4.1.4.1	Asociar un valor con una clave:.....	24
3.4.1.4.2	Obtención de un valor asociado a una clave especificada:.....	24

3.4.1.4.3	Comprobación de si el mapa contiene una clave especificada:.....	24
3.4.1.4.4	Comprobación de si el mapa contiene un valor especificado:.....	25
3.4.1.4.5	Eliminación de una asignación del mapa:.....	25
3.4.1.4.6	Sustitución de un valor asociado a una clave especificada:.....	25
3.4.1.4.7	Obtención del tamaño del mapa:.....	26
3.4.1.4.8	Comprobación de si el mapa está vacío:.....	26
3.4.1.5	Iteración sobre un mapa (utilizando vistas de colección).....	26
3.4.1.6	Realización de operaciones masivas con mapas.....	28
3.4.1.7	Mapas simultáneos.....	29
3.5	Cola.....	29
3.5.1.1	Características de la cola.....	30
3.5.1.2	Comportamientos de la cola.....	30
3.5.1.3	Interfaces de cola.....	30
3.5.1.4	Implementaciones de la cola principal.....	31
3.5.1.4.1	Implementaciones de cola de uso general:.....	31
3.5.1.4.2	Implementaciones simultáneas de colas:.....	31
3.5.1.5	Comprender la estructura de api de cola.....	31
3.5.1.5.1	Comprender la estructura api de la interfaz de cola.....	31
3.5.1.5.2	Comprensión de la estructura api de la interfaz de Deque.....	32
3.5.1.5.3	Comprensión de la estructura api de la interfaz BlockingQueue.....	32
3.5.1.5.4	Comprensión de la estructura api de la interfaz BlockingDeque.....	32
3.5.1.6	Realización de operaciones en colecciones de colas.....	33
3.5.1.6.1	Creación de una nueva instancia de cola.....	33
3.5.1.6.2	Adición de nuevos elementos a la cola.....	34
3.5.1.6.3	Extracción de la cabeza de la cola.....	35
3.5.1.6.4	Examinar la cabeza de la cola.....	36
3.5.1.7	Iteración sobre elementos en la cola.....	37
3.5.1.8	Colas simultáneas.....	38

Colecciones de Java

1 Introducción

Un tema que aparece frecuentemente en la programación es el de organizar una colección de objetos en memoria. Este tema también es conocido con el título “estructuras de datos”, y se relaciona también a distintos algoritmos para agilizar el acceso. Antes de hablar específicamente de las colecciones de Java voy a desviarme un rato hablando un poco sobre el problema en general de almacenar colecciones de datos en memoria.

2 Algoritmos y estructuras de datos

Todo programador aprende, apenas empieza, un par de estructuras de datos simples. Podríamos decir que la primer estructura de datos es la variable. Almacena un solo objeto, y el tiempo de acceso es ínfimo. La segunda estructura de datos que siempre se aprende es el arreglo (en inglés “array”). Esta colección nos permite obtener un valor dado una posición en una tabla. Es muy veloz ya que esta estructura tiene un fuerte paralelo con cómo se organiza internamente la memoria de una computadora. Los valores en un arreglo están en orden, y en ese mismo orden están dispuestos en la memoria, por lo tanto el acceso lo maneja directamente el hardware (porque la memoria funciona físicamente como un arreglo).

Un arreglo en Java se hace así (pero asumamos que ya se sabe).

```
String[] nombres = new String[10];
```

Muchos programadores se detienen aquí. Ayuda a esto el hecho de que en algunos lenguajes el arreglo era la estructura de datos más compleja soportada (por ejemplo en BASIC... ¡o incluso C!).

Uno puede usar arreglos para todo. ¿Quiero guardar la lista de helados que me gusta? ¿Las notas que se sacaron los alumnos de una clase? ¿La colección de coordenadas que forman un mapa? Puede guardarse en un arreglo... ahora... supongamos que quiero guardar las notas de los alumnos de una clase. Si quiero usar un arreglo podría actuar así: Creo una clase con el nombre del alumno y su nota:

```
class Nota
{
    String alumno;
    int nota;

    Nota(String alumno, int nota) {
        this.alumno = alumno;
        this.nota = nota;
    }

    public int getNota() { return nota; }
    public String getAlumno() { return alumno; }
}
```

Ahora almaceno las notas de 500 alumnos:

```
Nota[] notas = new Nota[500];

notas[0] = new Nota("Juan", 7);
notas[1] = new Nota("Pedro", 8);
notas[2] = new Nota("Manuel", 4);
notas[3] = new Nota("David", 6);

// ...

notas[499] = new Nota("Elías", 9);
```

Ahora... ¿Qué pasa si queremos averiguar la nota de Elías? No sabemos "a priori" en qué posición del arreglo está guardada la nota. La única manera que tenemos de proceder es simplemente recorrer el arreglo y preguntar si ya llegamos al alumno que queremos:

```
for(Nota n : notas)
    if(n.getNombre().equals("Elías"))
        return n.getNota();
```

¡Esto es lentísimo! Está bien, “Elías” es el peor caso, ya que se encuentra último. A Elías lo encontraremos después de haber efectuado 499 comprobaciones. Pero si suponemos que todos los alumnos son accedidos más o menos con una misma probabilidad tenemos que el número de comprobaciones promedio que se necesitarán por búsqueda de alumno es de 250!

¿Podemos hacerlo mejor? ¿Qué tal si pudiéramos contar con que el orden de las notas coincide con el orden alfabético del nombre de los alumnos? En ese caso podríamos ir al alumno 250, y preguntar: ¿Elías es mayor o menor alfabéticamente que vos? Si el alumno 250 es Luis, Elías claramente está antes que él. Luego, Elías está entre el 0 y el 249. Entonces vamos a comprobar al alumno 125, que es, por ejemplo, “Carlos”. Elías está después que Carlos, entonces sabemos que su posición es mayor a 125 (y era menor a 249). Así vamos acotando hasta llegar a Elías en, a lo sumo, 9 comprobaciones. A este algoritmo se le suele llamar “búsqueda dicotómica”.

No hace falta entender del todo el párrafo anterior. El quid de la cuestión es entender que, mediante algoritmos, redujimos la necesidad de comprobar 250 veces en promedio, a comprobar a lo sumo nueve veces. Y si en vez de 500 nombres tenemos 100.000 necesitaremos comprobar, a lo sumo, 17 veces. El uso de este algoritmo depende del hecho de que los datos que ordenamos están ordenados.

Asimismo cada una de las soluciones puede traer problemas secundarios que serán más o menos importantes de acuerdo a nuestras necesidades. En el ejemplo anterior, por ejemplo, necesitamos mantener un arreglo ordenado... ¿qué pasa si se agrega un nuevo alumno? Habrá que insertarlo en la posición correcta de acuerdo al orden alfabético y desplazar a todos lo que queden a su derecha un lugar. En el peor caso es recorrer todo el arreglo!

Todo esto quiere decir que el tipo de estructura de datos que se use dependerá de lo que puede asumir de la colección almacenada y de cómo se espera que esa colección sea usada (cómo sea leída y cómo sea modificada). Algunas preguntas a hacerse son:

- ¿Me interesa mantener el orden arbitrario original? ¿Puedo (como con las notas) reordenar los ítems?
- ¿Hay duplicados?
- ¿Con qué frecuencia es necesario añadir nuevos elementos?

3 Colecciones en Java

Java tiene desde la versión 1.2 todo un juego de clases e interfaces para guardar colecciones de objetos. En él, todas las entidades conceptuales están representadas por interfaces, y las clases se usan para proveer implementaciones de esas interfaces. Una introducción conceptual debe entonces enfocarse primero en esas interfaces.

La interfaz nos dice qué podemos hacer con un objeto. Un objeto que cumple determinada interfaz es “algo con lo que puedo hacer X”. La interfaz no es la descripción entera del objeto, solamente un mínimo de funcionalidad con la que debe cumplir.

Como corresponde a un lenguaje tan orientado a objetos, estas clases e interfaces están estructuradas en una jerarquía. A medida que se va descendiendo a niveles más específicos aumentan los requerimientos y lo que se le pide a ese objeto que sepa hacer.

3.1 Interface Collection

La interfaz más importante es Collection. Una Collection es todo aquello que se puede recorrer (o “iterar”) y de lo que se puede saber el tamaño. Muchas otras clases extenderán Collection imponiendo más restricciones y dando más funcionalidades. Es de notar que el requisito de “que se sepa el tamaño” hace inconveniente utilizar estas clases con colecciones de objetos de las que no se sepa “a priori” la cantidad (ésto podría considerarse una limitación de este framework).

Por las dudas vuelvo a aclarar: No puedo construir una Collection. No se puede hacer “new” de una Collection, sino que todas las clases que realmente manejan colecciones “son” Collection, y admiten sus operaciones.

Las operaciones básicas de una collection entonces son:

add(T)

Añade un elemento.

iterator()

Obtiene un “iterador” que permite recorrer la colección visitando cada elemento una vez.

size()

Obtiene la cantidad de elementos que esta colección almacena.

contains(t)

Pregunta si el elemento t ya está dentro de la colección.

Si yo tengo un objeto Collection hay muchas cosas que no puedo asumir. No puedo asumir que el orden en el que lo recorro sea relevante, es decir, que si lo recorro de nuevo vea los elementos en el mismo orden en el que los vi la primera vez. Tampoco puedo asumir que no hay duplicados. No puedo asumir características de rendimiento: Preguntar si existe un objeto en la colección puede tardar desde muy poco hasta... mucho =).

Una capacidad de un objeto Collection es la de poder ser recorrido. Como a este nivel no está definido un orden, la única manera es proveyendo un iterador, mediante el método iterator(). Un iterador es un objeto “paseador” que nos permite ir obteniendo todos los objetos al ir invocando progresivamente su método next(). También, si la colección es modificable, podemos remover un objeto durante el recorrido mediante el método remove() del iterador. El siguiente ejemplo recorre una colección de Integer borrando todos los ceros:

```
void borrarCeros(Collection<Integer> ceros)
{
    Iterator<Integer> it = ceros.iterator();
    while(it.hasNext())
    {
        int i = it.next();
        if(i == 0)
            it.remove();
    }
}
```

En este ejemplo hago uso de la conversión automática entre Integer e int.

A partir de Java 6 hay una manera simplificada de recorrer una collection (que sirve si no necesitamos borrar elementos). Se hace mediante un nuevo uso del keyword for:

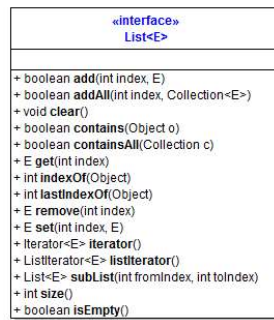
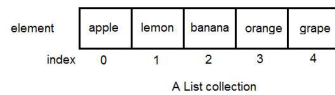
```
void mostrar(Collection<?> col)
{
    for(Object o : col)
        System.out.println(o);
}
```

Internamente este código no hace otra cosa que obtener el iterador, pero queda mucho más elegante y legible de esta manera. Pero al mismo tiempo, el no disponer del iterador explícitamente hace imposible usar esta sintaxis para hacer lo que hice en el ejemplo previo: ir borrando elementos.

Hay cuatro interfaces que extienden Collection: List, Set, Map y Queue. Cada una de ellas agrega condiciones sobre los datos que almacena y como contrapartida ofrece más funcionalidad. A continuación cuento de qué se trata cada una de ellas.

3.2 Interface List

Un List, o simplemente lista, es una Collection. La diferencia que tiene una lista con una Collection es que la lista mantiene un orden arbitrario de los elementos y permite acceder a los elementos por orden. Podríamos decir que en una lista, por lo general, el orden es dato. Es decir, el orden es información importante que la lista también nos está almacenando.



No hay ningún método en Collection para obtener el tercer elemento. No lo puede haber porque, como se dijo, a nivel Collection ni siquiera estamos seguros de que si volvemos a recorrer la colección los elementos aparecerán en el mismo orden. Una lista sí debe permitir acceder al tercer elemento, por eso se añaden los siguientes métodos:

get(int i)

Obtiene el elemento en la posición i.

set(int i, T t)

Pone al elemento t en la posición i.

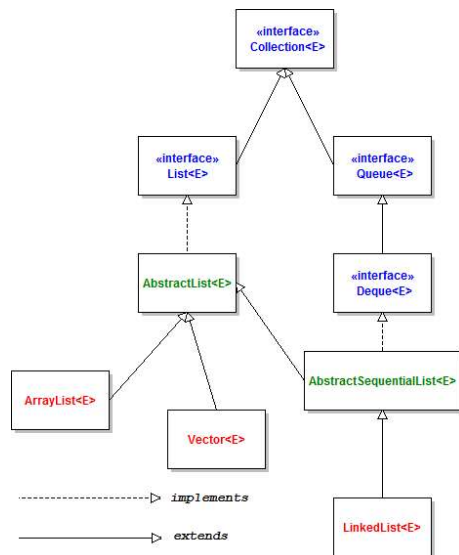
Existen en Java principalmente dos implementaciones de List, las dos útiles en distintos casos. Una de ellas es casi igual a los arreglos comunes (como el de notas que usaba más arriba para ejemplificar). Esta implementación es ArrayList. La ventaja de ArrayList por sobre un array común es que es expansible, es decir que crece a medida que se le añaden elementos (mientras que el tamaño de un array es fijo desde su creación). Lo bueno es que el tiempo de acceso a un elemento en particular es ínfimo. Lo malo es que si queremos eliminar un elemento del principio, o del medio, la clase debe mover todos los que le siguen a la posición anterior, para “tapar” el agujero que deja el elemento removido. Esto hace que sacar elementos del medio o del principio sea costoso.

La otra implementación es LinkedList (lista enlazada). En ésta, los elementos son mantenidos en una serie de nodos atados entre sí como eslabones de una cadena. Cada uno de estos nodos apunta a su antecesor y al elemento que le sigue. Nada más. No hay nada en cada uno de esos nodos que tenga algo que ver con la posición en la lista. Para obtener el elemento número “n”, esta implementación de List necesita entonces empezar desde el comienzo, desde el primer nodo, e ir avanzando al “siguiente” n veces. Buscar el elemento 400 entonces implica 400 de esos pasitos. La ventaja es que es posible eliminar elementos del principio de la lista y del medio de manera muy eficiente. Para eliminar un elemento solamente hay que modificar a sus dos “vecinos” para que se “conecten” entre sí ignorando al elemento que se está borrando. Como en una cadena, se retira un eslabón abriendo los eslabones adyacentes al que se eliminó y cerrándolos de modo que lo excluyan. No es necesario hacerle ningún cambio al resto de los elementos de la lista.

En otros lenguajes lidiar con listas enlazadas puede ser un poco más trabajoso. En Java, LinkedList se usa exactamente igual que otros tipos de List, por lo que no hay que saber nada adicional para empezar a usarla. Bueno, esto no es del todo cierto... hay que tener muy en claro sus particularidades en cuanto a rendimiento. Su método get(int) es particularmente lento porque, como dije, necesita recorrer para llegar al elemento pedido. Esto hace que recorrer la lista con un simple for(int i = 0 ; i < lista.size(); i++) sea tremendamente lento! La complejidad pasa de ser lineal a cuadrática, es decir: Si se recorre así una lista de 300 elementos, se tarda como si tuviera 44.850 elementos! Una LinkedList sólo debe recorrerse mediante [iteradores](#).

Un uso ideal de LinkedList es para la creación de “colas”, en las que los elementos se añaden al final, y se eliminan del comienzo. Para este uso se puede usar, en vez de List, la interfaz [Queue](#) (también implementada por LinkedList) que es más específica para esta tarea.

Además de ArrayList y LinkedList, la clase Vector es una colección heredada y se ha modernizado para implementar el interfaz List. Vector es seguro para subprocesos, pero ArrayList y LinkedList no lo son porque no está sincronizada. El siguiente diagrama de clases representa el árbol de herencia de las colecciones de tipo List :



A continuación se muestra un ejemplo rápido de creación de un nuevo ArrayList y un LinkedList para contener objetos de tipo String; añadir algunos elementos a ellos; y luego imprimir las colecciones:

- Ejemplo rápido de ArrayList:

```

1List<String> listStrings = new ArrayList<String>();
2listStrings.add("One");
3listStrings.add("Two");
4listStrings.add("Three");
5listStrings.add("Four");
6System.out.println(listStrings);
  
```

- Ejemplo rápido de LinkedList:

```

1List<String> listStrings = new LinkedList<String>();
2listStrings.add("Five");
3listStrings.add("Six");
4listStrings.add("Seven");
5listStrings.add("Eight");
6System.out.println(listStrings);
  
```

Los ejemplos de código de este tutorial giran en las dos implementaciones comunes: ArrayList y LinkedList.

3.2.1.1 Creación de una nueva lista

Es una buena práctica declarar una instancia de lista con un parámetro de tipo genérico, por ejemplo:

```

1List<Object> listAnything = new ArrayList<Object>();
2List<String> listWords = new ArrayList<String>();
3List<Integer> listNumbers = new ArrayList<Integer>();
4List<String> linkedWords = new LinkedList<String>();
  
```

Desde Java 7, podemos eliminar el parámetro type en el lado derecho de la siguiente manera:

```

1List<Integer> listNumbers = new ArrayList<>();
2List<String> linkedWords = new LinkedList<>();
  
```

El compilador puede [inferir el parámetro de tipo real](#) desde la declaración en el lado izquierdo. Desde Java 9, puede crear una colección List a partir de un conjunto fijo de elementos utilizando el método de fábrica List.of(e1, e2, e3...). por ejemplo:

```
1List<Integer> listNumbers = List.of(1, 2, 3, 4, 5, 6);
```

Tenga en cuenta que la colección List devuelta por el método de fábrica List.of() es inmutable, lo que significa que no puede agregarle más elementos. Desde Java 10, puede acortar la declaración de una colección List utilizando la palabra reservada var como esta:

```
1var employees = new ArrayList<Employee>();
```

El compilador puede inferir el tipo de la variable a la izquierda en función del tipo de objeto en el lado derecho. Y var solo se puede utilizar para declarar variables locales. Al crear un nuevo ArrayList mediante el constructor vacío, la lista se construye con una capacidad inicial de diez. Si está seguro de cuántos elementos se agregarán a la lista, se recomienda especificar una capacidad lo suficientemente grande. Supongamos que, si sabemos que una lista contiene alrededor de 1000 elementos, declare la lista de la siguiente manera:

```
1List<Integer> listNumbers = new ArrayList<>(1000);
```

También es posible construir una lista que tome elementos de una colección existente, por ejemplo:

```
1List<Integer> listNumberOne; // existing collection
2List<Integer> listNumberTwo = new ArrayList<>(listNumberOne);
```

La listNumberTwo construida con copias de todos los elementos de la listNumberOne.

3.2.1.2 Operaciones básicas de la lista: agregar, recuperar, actualizar, eliminar elementos

3.2.1.2.1 Adición de elementos a una lista:

Los métodos add(Object), add(index, Object) y addAll() se utilizan para agregar elementos a la lista. Requiere agregar elementos del mismo tipo (o sub tipo) que el parámetro type declarado por la lista. por ejemplo:

```
1List<String> listStrings = new ArrayList<String>();
2// OK to add Strings:
3listStrings.add("One");
4listStrings.add("Two");
5listStrings.add("Three");
6// But this will cause compile error
7listStrings.add(123);
```

Adición de elementos de sub tipos del tipo declarado:

```
1List<Number> linkedNumbers = new LinkedList<>();
2linkedNumbers.add(new Integer(123));
3linkedNumbers.add(new Float(3.1415));
4linkedNumbers.add(new Double(299.988));
5linkedNumbers.add(new Long(67000));
```

Podemos insertar un elemento en la lista en un índice especificado, por ejemplo:

```
1listStrings.add(1, "Four");
```

Eso inserta la cadena "Cuatro" en la 2ª posición de la lista. También podemos agregar todos los elementos de una colección existente al final de la lista:

```
1listStrings.addAll(listWords);
```

O agregue los elementos a la lista en una posición especificada:

```
1listStrings.addAll(2, listWords);
```

Eso inserta todos los elementos de la colección listWords en 3ª posición de la colección listStrings.

3.2.1.2.2 Recuperar elementos de una lista

El método get() se utiliza para recuperar un elemento de la lista en un índice especificado. Por ejemplo, el código siguiente obtiene un elemento en 2

Nº posición en la lista de matrices y un elemento en 4º posición en la lista vinculada:

```
1String element = listStrings.get(1);
2Number number = linkedNumbers.get(3);
```

Para una implementación de LinkedList, podemos obtener el primer y el último elemento como este:

```
1LinkedList<Number> numbers = new LinkedList<Number>();
2// add elements to the list...
3// get the first and the last elements:
4Number first = numbers.getFirst();
5Number last = numbers.getLast();
```

Tenga en cuenta que los métodos getFirst() y getLast() son específicos de la clase LinkedList. por ejemplo:

```
1listStrings.set(2, "Hi");
```

Eso reemplaza a los 3ª elemento de la lista por la nueva cadena "Hola".

3.2.1.2.3 Eliminación de elementos de una lista

Para quitar un elemento de la lista, utilice el método remove(index) o remove(Object) que quita el elemento en el índice especificado o por referencia de objeto. por ejemplo:

- Retire el elemento en el tercera posición en la lista:

```
1listStrings.remove(2);
```

Si el índice especificado está fuera de rango (índice < 0 o tamaño de lista index >=), se produce una excepción java.lang.IndexOutOfBoundsException.

- Quite el elemento String "Two" de la lista:

```
1listStrings.remove("Two");
```

Notas sobre el método remove(Object):

- Compara el objeto especificado con los elementos de la lista mediante su método equals(), por lo que si utiliza su propio tipo de objeto definido, asegúrese de implementar correctamente el método equals().
 - Solo quita la primera aparición del elemento especificado en la lista (es decir, si una lista contiene elementos duplicados, solo se quita el primer elemento).
 - Devuelve true si la lista contenía el elemento especificado o false de otro modo. Por lo tanto, se recomienda comprobar el valor devuelto de este método, por ejemplo:

```
1if (listStrings.remove("Ten")) {
```

```

2    System.out.println("Removed");
3} else {
4    System.out.println("There is no such element");
5}

```

Para eliminar todos los elementos de la lista, utilice el método `clear()`:

```
1listStrings.clear();
```

3.2.1.3 Iteración sobre los elementos de una lista

Básicamente, podemos utilizar el bucle mejorado para recorrer en iteración todos los elementos de la lista, de la siguiente manera:

```

1for (String element : listStrings) {
2    System.out.println(element);
3}

```

O utilice un iterador como este:

```

1Iterator<String> iterator = listStrings.iterator();
2while (iterator.hasNext()) {
3    System.out.println(iterator.next());
4}

```

Para obtener más información específica de la lista, utilice un iterador de lista como se muestra a continuación:

```

1Iterator<Number> iterator = linkedNumbers.listIterator();
2while (iterator.hasNext()) {
3    System.out.println(iterator.next());
4}

```

Desde Java 8, podemos utilizar el método `forEach()` como este:

```
1listStrings.forEach(s -> System.out.println(s));
```

Para obtener más detalles y ejemplos, consulte el tutorial: [Java Collections Looping Ejemplo](#) Para obtener más información sobre el método de iteración `forEach`, consulte el tutorial: Los [4 métodos para iterar colecciones en Java](#)

3.2.1.4 Búsqueda de un elemento en una lista

Para buscar la posición de un elemento específico en la lista o para saber si la lista contiene el elemento especificado, se pueden utilizar los métodos siguientes:

- **`boolean contains(Object)`** : devuelve true si la lista contiene el elemento especificado.
 - **`int indexOf(Object)`** : devuelve el índice de la primera aparición del elemento especificado en la lista o -1 si no se encuentra el elemento.
 - **`int lastIndexOf(Object)`** : devuelve el índice de la última aparición del elemento especificado en la lista o -1 si no se encuentra el elemento.

Ejemplos:

```

1if (listStrings.contains("Hello")) {
2    System.out.println("Found the element");
3} else {
4    System.out.println("There is no such element");
5}
6int firstIndex = linkedNumbers.indexOf(1234);
7int lastIndex = listStrings.indexOf("Hello");

```

Tenga en cuenta que los métodos anteriores comparan los elementos utilizando su método equals(), por lo que si define su propio tipo, asegúrese de que implementa correctamente el método equals().

3.2.1.5 Ordenación de una lista

Antes de Java 8, la forma más sencilla de ordenar los elementos de una lista es utilizando el método estático Collections.sort() que ordena la lista especificada en orden ascendente, en función del orden natural de sus elementos. Este es un ejemplo:

```

1List<String> listStrings = new ArrayList<String>();
2listStrings.add("D");
3listStrings.add("C");
4listStrings.add("E");
5listStrings.add("A");
6listStrings.add("B");
7System.out.println("listStrings before sorting: " + listStrings);
8Collections.sort(listStrings);
9System.out.println("listStrings after sorting: " + listStrings);

```

salida:

```

1listStrings before sorting: [D, C, E, A, B]
2listStrings after sorting: [A, B, C, D, E]

```

Tenga en cuenta que todos los elementos de la lista deben implementar la interfaz comparable, por lo que si define su propio tipo, asegúrese de implementar esa interfaz y su método compareTo(). Desde Java 8, la interfaz List introduce el método sort(), por lo que puede ordenar los elementos en un ArrayList o LinkedList directamente de la siguiente manera:

```

1listStrings.sort(null); // sort by natural ordering of the elements

```

Para obtener más detalles y ejemplos, consulte el artículo: [Ordenar ejemplos de colecciones de listas](#)

3.2.1.6 Copiar elementos de una lista en otra

El método estático Collections.copyList(dest, src) nos permite copiar todos los elementos de la lista de origen en el de destino. Tenga en cuenta que la lista de destino debe ser lo suficientemente grande como para contener toda la lista de origen. Este es un ejemplo:

```

1List<String> sourceList = new ArrayList<String>();
2sourceList.add("A");
3sourceList.add("B");
4sourceList.add("C");
5sourceList.add("D");
6List<String> destList = new ArrayList<String>();
7destList.add("V");
8destList.add("W");
9destList.add("X");
10destList.add("Y");
11destList.add("Z");
12System.out.println("destList before copy: " + destList);
13Collections.copy(destList, sourceList);
14System.out.println("destList after copy: " + destList);

```

La salida sería:

```

1destList before copy: [V, W, X, Y, Z]
2destList after copy: [A, B, C, D, Z]

```

3.2.1.7 Barajar una lista

Para permutar aleatoriamente los elementos de una lista, utilice el método estático **Collections.shuffle()**. Este es un ejemplo rápido:

```

1List<Integer> numbers = new ArrayList<Integer>();
2for (int i = 0; i <= 10; i++) numbers.add(i);
3System.out.println("List before shuffling: " + numbers);
4Collections.shuffle(numbers);
5System.out.println("List after shuffling: " + numbers);

```

La salida sería:

```

1List before shuffling: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
2List after shuffling: [6, 4, 5, 0, 1, 3, 9, 7, 2, 10, 8]

```

3.2.1.8 Invertir elementos en una lista

Para invertir el orden de los elementos de una lista, utilice el método estático **Collections.reverse()**. Este es un ejemplo rápido:

```

1List<Integer> numbers = new ArrayList<Integer>();
2for (int i = 0; i <= 10; i++) numbers.add(i);
3System.out.println("List before reversing: " + numbers);
4Collections.reverse(numbers);
5System.out.println("List after reversing: " + numbers);

```

La salida sería:

```

1List before reversing: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
2List after reversing: [10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0]

```

3.2.1.9 Extracción de una porción de una lista

El `subList(fromIndex, toIndex)` nos permite obtener una parte de la lista entre el especificado `fromIndex`(inclusive) y `toIndex`(exclusivo). Este es un ejemplo:

```
1 List<String> listNames = Arrays.asList("Tom", "John", "Mary", "Peter",  
2 "David", "Alice");  
3 System.out.println("Original list: " + listNames);  
4 List<String> subList = listNames.subList(2, 5);  
5 System.out.println("Sub list: " + subList);
```

salida:

```
1Original list: [Tom, John, Mary, Peter, David, Alice]  
2Sub list: [Mary, Peter, David]
```

Tenga en cuenta que la sub lista es solo una vista de la lista original, por lo que las modificaciones realizadas en la lista original se reflejarán en la sublista.

3.2.1.10 Conversión entre listas y matrices

Java Collection Framework nos permite convertir fácilmente entre listas y matrices. El `Arrays.asList(T... a)` método convierte una matriz de tipo `T` en una lista de tipo `T`. Este es un ejemplo:

```
1 List<String> listNames = Arrays.asList("John", "Peter", "Tom", "Mary",  
2 "David", "Sam");  
3 List<Integer> listNumbers = Arrays.asList(1, 3, 5, 7, 9, 2, 4, 6, 8);  
4 System.out.println(listNames);  
5 System.out.println(listNumbers);
```

salida:

```
1[John, Peter, Tom, Mary, David, Sam]  
2[1, 3, 5, 7, 9, 2, 4, 6, 8]
```

Y la interfaz `List` proporciona el método `toArray()` que devuelve una matriz de Objetos que contiene todos los elementos de la lista en secuencia adecuada (del primer al último elemento). Este es un ejemplo:

```
1List<String> listWords = new ArrayList<String>();  
2// add elements to the list  
3Object[] arrayWords = listWords.toArray();
```

Y el método `toArray(T[] a)` devuelve una matriz de tipo `T`, por ejemplo:

```
1String[] words = listWords.toArray(new String[0]);  
2Integer[] numbers = listNumbers.toArray(new Integer[0]);
```

Tenga en cuenta que la matriz devuelta contiene copias de elementos de la lista, lo que significa que podemos modificar la matriz de forma segura sin afectar a la lista.

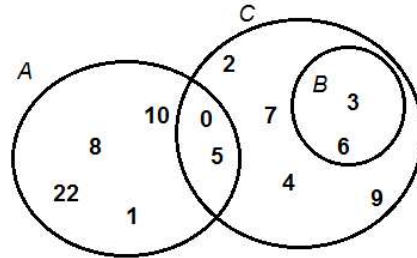
3.2.1.11 Video para la práctica de Listas

Para la práctica en la colección de listas Java, le recomiendo que vea este tutorial de vídeo:

[Video para la práctica de Listas](#)

3.3 Set

Básicamente, Set es un tipo de colección que no permite elementos duplicados. Esto significa que un elemento solo puede existir una vez en un conjunto. Modela la abstracción establecida en matemáticas. La siguiente imagen ilustra tres conjuntos de números en matemáticas:



Características de la Colección Set

Las siguientes características diferencian una colección Set de otras del [marco de Java Collections](#):

- No se permiten elementos duplicados.
- Los elementos no se almacenan en orden. Esto significa que no puede esperar que los elementos se ordenen en ningún orden al iterar sobre los elementos de un conjunto.

Cuando usar la Colección Set:

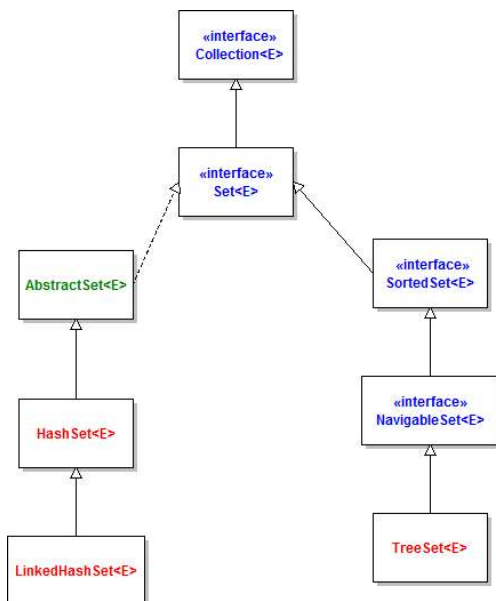
En función de las características, considere la posibilidad de utilizar una colección Set cuando:

- Desea almacenar elementos claramente sin duplicación o sea elementos únicos.
- No te importa el orden de los elementos.

Por ejemplo, puede usar un conjunto para almacenar números enteros únicos; puedes usar un Set para almacenar cartas al azar en un juego de cartas; puede utilizar un conjunto para almacenar números en orden aleatorio, etc.

Implementaciones de Set:

Java proporciona tres implementaciones principales del interfaz Set: HashSet, LinkedHashSet y TreeSet. El siguiente diagrama de clases representa el árbol de herencia de las colecciones de tipo Set :



Los Sets (y los Maps) aprovechan cierta cosa característica de Java: Todos los objetos heredan de Object, por lo tanto todos los métodos de la clase Object están presentes en todos los objetos. Dicho de otra manera, hay ciertas cosas que todo objeto en Java sabe hacer. Dos de estas cosas son:

- Saber si es igual a otro, con su método equals().
- Devolver un número entero de modo tal que si dos objetos son iguales ese número también lo será (se conoce esto como un hash). Esto todo objeto lo sabe hacer con su método hashCode().

3.3.1.1 **HashSet:**

Es la implementación con mejor rendimiento y es una implementación set ampliamente utilizada. Representa las características principales de los conjuntos: sin duplicación y desordenado.

La clase HashSet aprovecha la segunda de las funciones. A cada objeto que se añade a la colección se le pide que calcule su “hash”. Este valor será un número entre -2147483647 y 2147483648. Basado en ese valor se lo guarda en una tabla. Más tarde, cuando se pregunta con contains() si un objeto x ya está, habrá que saber si está en esa tabla. ¿En qué posición de la tabla está? HashSet puede saberlo, ya que para un objeto determinado, el hash siempre va a tener el mismo valor. Entonces la función contains de HashSet saca el hash del objeto que le pasan y va con eso a la tabla. En la posición de la tabla hay una lista de objetos que tienen ese valor de hash, y si uno de esos es el buscado contains devuelve true.

Un efecto de este algoritmo es que el orden en el que aparecen los objetos al recorrer el set es impredecible. También es importante darse cuenta de que es crítico que la función hashCode() tiene que devolver siempre el mismo valor para los objetos que se consideran iguales (o sea que equals() da true). Si esto no es así, HashSet pondrá al objeto en una posición distinta en la tabla que la que más adelante consultará cuando se llame a contains, y entonces contains dará siempre falso, por más que se haya hecho correctamente el add. Esto mismo puede suceder si se usan como claves objetos que varíen.

3.3.1.2 **LinkedHashSet:**

Esta implementación ordena sus elementos en función del orden de inserción. Así que considere la posibilidad de usar un **LinkedHashSet** cuando desee almacenar elementos únicos en orden.

3.3.1.3 **TreeSet:**

esta implementación ordena sus elementos en función de sus valores, ya sea por su ordenación natural o por un comparador proporcionado en tiempo de creación.

Los objetos con independencia de cómo y dónde sean usados deben saber ordenarse. A diferencia de “equals” y “hashCode”, que están en todos los objetos, la capacidad de “ordenarse” está sólo en aquellos que implementan la interfaz Comparable. Cuando un objeto implementa esta interfaz promete saber compararse con otros (con el método compareTo()), y responder con este método si él está antes, después o es igual al objeto que se le pasa como parámetro. Al orden resultante de usar este método se le llama en Java “orden natural”. Muchas de las clases de Java implementan Comparable, por ejemplo String lo hace, definiendo un orden natural de los strings que es el obvio, el alfabético. También implementan esta interfaz Date, Number, etc. y los “órdenes naturales” que definen estas implementaciones son los que uno esperaría.

Si yo creo una clase mía llamada Alumno, queda a mi cargo, si así lo quiero, la definición de un orden natural para los alumnos. Puedo elegir usar el apellido, el nombre, el número de matrícula, etc. De acuerdo al atributo que elija para definir el orden natural codificaré el método compareTo(). Lo que es importante es que la definición de este método sea compatible con el equals(); esto es que a.equals(b) si y sólo si a.compareTo(b) == 0.

TreeSet usa una técnica completamente diferente a la explicada para HashSet. Construye un árbol con los objetos que se van agregando al conjunto. Un árbol es una forma en computación de tener un conjunto de cosas todo el tiempo en orden, y permitir que se agreguen más cosas y que el orden se mantenga. Al tener todo en orden TreeSet puede fácilmente saber si un objeto está o no, por medio de una técnica muy similar a la explicada al comienzo del tema para buscar los nombres de las notas.

Una ventaja de TreeSet es que el orden en el que aparecen los elementos al recorrerlos es el orden natural de ellos (los objetos deberán implementar Comparable, como lo explico arriba; si no lo hacen se deberá especificar una función de comparación manualmente). Una desventaja es que mantener todo ordenado tiene un costo, y esta clase es un poquito menos eficiente que HashSet.

3.3.1.4 Creación de un nuevo conjunto

Utilice siempre [genéricos](#) para declarar un conjunto de tipos específicos, por ejemplo, un conjunto de números enteros:

```
1Set<Integer> numbers = new HashSet<>();
```

Recuerde el uso del tipo de interfaz (Establecer) en como tipo de referencia, y la implementación concreta (HashSet, LinkedHashSet, TreeSet, etc) como el tipo de objeto real:

```
1Set<String> names = new LinkedHashSet<>();
```

Podemos crear un set a partir de una colección existente. Este es un truco para eliminar elementos duplicados en la colección non-Set. Considere el siguiente fragmento de código:

```
1List<Integer> listNumbers = Arrays.asList(3, 9, 1, 4, 7, 2, 5, 3, 8, 9, 1, 3,  
28, 6);  
2System.out.println(listNumbers);  
3Set<Integer> uniqueNumbers = new HashSet<>(listNumbers);  
4System.out.println(uniqueNumbers);
```

salida:

```
1[3, 9, 1, 4, 7, 2, 5, 3, 8, 9, 1, 3, 8, 6]  
2[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Verá, la lista listNumbers contiene números duplicados y el conjunto uniqueNumbers quita los duplicados. Al igual que con Java 8, podemos utilizar stream con funciones de filtro y colección para devolver un Set de una colección. El código siguiente recopila solo números impares en un Conjunto de la listaNumbers anterior:

```
1Set<Integer> uniqueOddNumbers = listNumbers.stream()  
2    .filter(number -> number % 2 != 0).collect(Collectors.toSet());  
3  
4System.out.println(uniqueOddNumbers);
```

salida:

```
1[1, 3, 5, 7, 9]
```

Tenga en cuenta que la capacidad predeterminada inicial de un HashSet y LinkedHashSet es 16, por lo que si está seguro de que el conjunto contiene más de 16 elementos, es mejor especificar una capacidad en el constructor. por ejemplo:

```
1Set<String> bigNames = new HashSet<>(1000);
```

Esto crea un nuevo HashSet con capacidad inicial es de 1000 elementos.

3.3.1.5 Realizar operaciones básicas en un conjunto

3.3.1.5.1 Adición de elementos a un conjunto:

El método `add()` devuelve `true` si el conjunto no contiene el elemento especificado y devuelve `false` si el conjunto ya contiene el elemento especificado:

```
1Set<String> names = new HashSet<>();
2names.add("Tom");
3names.add("Mary");
4
5if (names.add("Peter")) {
6    System.out.println("Peter is added to the set");
7}
8
9if (!names.add("Tom")) {
10    System.out.println("Tom is already added to the set");
11}
```

salida:

```
1Peter is added to the set
2Tom is already added to the set
```

El Set puede contener un elemento null:

```
1names.add(null);
```

3.3.1.5.2 Eliminación de un elemento de un conjunto:

El método `remove()` quita el elemento especificado del conjunto si está presente (el método devuelve `true` o `false` de otro modo):

```
1if (names.remove("Mary")) {
2    System.out.println("Marry is removed");
3}
```

Tenga en cuenta que los objetos del Conjunto deben implementar los métodos `equals()` y `hashCode()` correctamente para que el Set pueda buscar y quitar los objetos.

3.3.1.5.3 Compruebe si un conjunto está vacío:

El método `isEmpty()` devuelve `true` si el conjunto no contiene ningún elemento, de lo contrario devuelve `false`:

```
1if (names.isEmpty()) {
2    System.out.println("The set is empty");
3} else {
4    System.out.println("The set is not empty");
5}
```

3.3.1.5.4 Elimine todos los elementos de un conjunto:

El método `clear()` elimina todos los elementos del conjunto. El conjunto estará vacío después:

```
1names.clear();
2
3if (names.isEmpty()) {
4    System.out.println("The set is empty");
5}
```

3.3.1.5.5 Obtenga un número total de elementos en un conjunto:

El método size() devuelve el número de elementos contenidos en el conjunto:

```
1Set<String> names = new HashSet<>();
2names.add("Tom");
3names.add("Mary");
4names.add("Peter");
5names.add("Alice");
6
7System.out.printf("The set has %d elements", names.size());
```

salida:

```
1The set has 4 elements
```

Tenga en cuenta que la interfaz Set no proporciona ninguna API para recuperar un elemento específico debido a su naturaleza de desorden. Excepto la implementación treeset permite recuperar el primer y el último elemento. Si desea sumergirse profundamente en el marco de colecciones Java, este [famoso libro de colección Java](#) es una buena lectura.

3.3.1.6 Iteración sobre elementos en un conjunto

3.3.1.6.1 Uso de un iterador:

```
1Set<String> names = new HashSet<>();
2names.add("Tom");
3names.add("Mary");
4names.add("Peter");
5names.add("Alice");
6
7
8Iterator<String> iterator = names.iterator();
9
10while (iterator.hasNext()) {
11    String name = iterator.next();
12    System.out.println(name);
13}
```

salida:

```
1Tom
2Alice
3Peter
4Mary
```

3.3.1.6.2 Uso del bucle for mejorado:

```
1for (String name : names) {
2    System.out.println(name);
3}
```

Uso del método `forEach()` con expresión lambda en Java 8:

```
1names.forEach(System.out::println);
```

Para obtener más información acerca de las colecciones mecanismo de iteración, vea: [Los 4 métodos para iterar colecciones en Java](#).

3.3.1.7 *Búsqueda de un elemento en un conjunto*

El `contains(Object)` método devuelve `true` si el conjunto contiene el elemento especificado, o devolver `false` de lo contrario. por ejemplo:

```
1Set<String> names = new HashSet<>();
2names.add("Tom");
3names.add("Mary");
4names.add("Peter");
5names.add("Alice");
6
7if (names.contains("Mary")) {
8    System.out.println("Found Mary");
9}
```

Tenga en cuenta que si el conjunto contiene objetos personalizados de su propio tipo, por ejemplo `Student` o `Employee`, el objeto debe implementar correctamente los métodos `equals()` y `hashCode()` para que el `Set` pueda encontrar los objetos.

3.3.1.8 *Realización de operaciones masivas entre dos conjuntos*

Podemos realizar algunas operaciones matemáticas entre dos conjuntos como subconjunto, unión, intersección y diferencia de conjunto. Supongamos que tenemos dos sets `s1` y `s2`.

3.3.1.8.1 Operación de subconjunto:

- `s1.containsAll(s2)` devuelve `true` si `s2` es un subconjunto de `s1` (`s2` es un subconjunto de `s1` si `s1` contiene todos los elementos de `s2`).

ejemplo:

```
1Set<Integer> s1 = new HashSet<>(Arrays.asList(20, 56, 89, 31, 8, 5));
2Set<Integer> s2 = new HashSet<>(Arrays.asList(8, 89));
3
4if (s1.containsAll(s2)) {
5    System.out.println("s2 is a subset of s1");
6}
```

salida:

```
1s2 is a subset of s1
```

3.3.1.8.2 Operación de unión:

- `s1.addAll(s2)` —`s1` se transforma en la unión de `s1` y `s2`. (La unión de dos conjuntos es el conjunto que contiene todos los elementos contenidos en cualquiera de los conjuntos.)

ejemplo:

```
1Set<Integer> s1 = new HashSet<>(Arrays.asList(1, 3, 5, 7, 9));
2Set<Integer> s2 = new HashSet<>(Arrays.asList(2, 4, 6, 8));
3
4System.out.println("s1 before union: " + s1);
5
6s1.addAll(s2);
7
8System.out.println("s1 after union: " + s1);
```

salida:

```
1s1 before union: [1, 3, 5, 7, 9]
2s1 after union: [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

3.3.1.8.3 Operación de intersección:

- `s1.retainAll(s2)` — `s1` se transforma en la intersección de `s1` y `s2` (La intersección de dos conjuntos es el conjunto que contiene sólo los elementos comunes a ambos conjuntos.)

ejemplo:

```
1Set<Integer> s1 = new HashSet<>(Arrays.asList(1, 2, 3, 4, 5, 7, 9));
2Set<Integer> s2 = new HashSet<>(Arrays.asList(2, 4, 6, 8));
3
4System.out.println("s1 before intersection: " + s1);
5
6s1.retainAll(s2);
7
8System.out.println("s1 after intersection: " + s1);
```

salida:

```
1s1 before intersection: [1, 2, 3, 4, 5, 7, 9]
2s1 after intersection: [2, 4]
```

3.3.1.8.4 Operación de diferencia:

- `s1.removeAll(s2)` — `s1` se transforma en la diferencia de conjunto (asimétrica) de `s1` y `s2`. (Por ejemplo, la diferencia establecida de `s1` menos `s2` es el conjunto que contiene todos los elementos que se encuentran en `s1` pero no en `s2`.)

ejemplo:

```
1Set<Integer> s1 = new HashSet<>(Arrays.asList(1, 2, 3, 4, 5, 7, 9));
2Set<Integer> s2 = new HashSet<>(Arrays.asList(2, 4, 6, 8));
3
4System.out.println("s1 before difference: " + s1);
5
6s1.removeAll(s2);
7
8System.out.println("s1 after difference: " + s1);
```

salida:

```
1s1 before difference: [1, 2, 3, 4, 5, 7, 9]
2s1 after difference: [1, 3, 5, 7, 9]
```

3.3.1.9 Conjuntos simultáneos

Las tres implementaciones HashSet, LinkedHashSet y TreeSet no están sincronizadas. Por lo tanto, si los usa en contexto simultáneo (multiprocesos), debe sincronizarlos externamente mediante el método estático Collections.synchronizedSet(). por ejemplo:

```
1Set<Integer> numbers = Collections.synchronizedSet(new HashSet<Integer>());
```

El conjunto devuelto está sincronizado (seguro para subprocesos). Y recuerde que debe sincronizar manualmente en el conjunto devuelto al iterar sobre él:

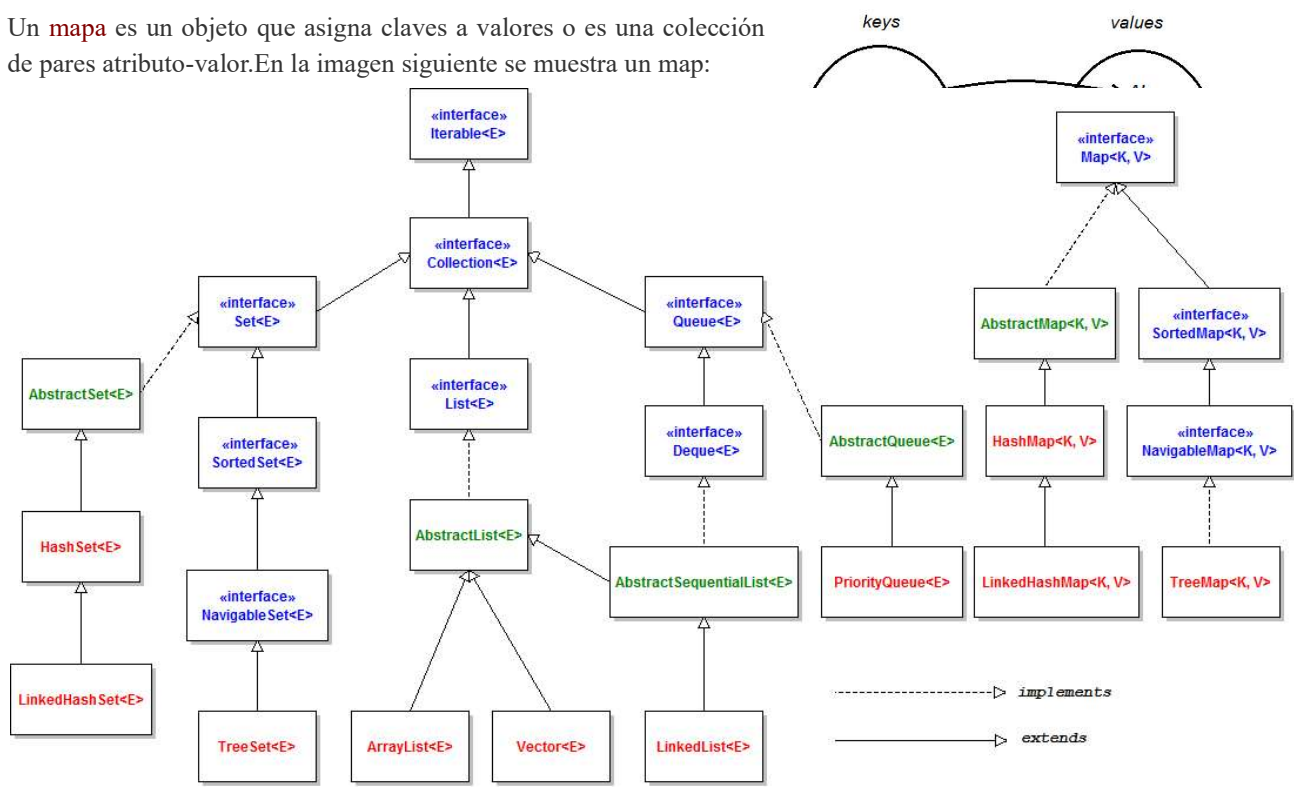
```
1synchronized (numbers) {
2    Iterator<Integer> iterator = numbers.iterator();
3
4    while (iterator.hasNext()) {
5        Integer number = iterator.next();
6        System.out.println(number);
7    }
8}
```

3.4 Map

Un Map representa lo que en otros lenguajes se conoce como “diccionario” y que se suele asociar a la idea de “tabla hash” (aunque no se implemente necesariamente con esa técnica). Un Map es un conjunto de valores, con el detalle de que cada uno de estos valores tiene un objeto extra asociado. A los primeros se los llama “claves” o “keys”, ya que nos permiten acceder a los segundos.

Cuando digo que las claves forman un conjunto, lo digo en el sentido Java: Son un “Set”, no puede haber duplicados. En otros lenguajes existen estructuras parecidas que admiten la existencia de claves duplicadas (a veces conocidos como “multimap”). La única manera de lograr esto en Java es haciendo que el map guarde listas de valores en vez de los valores sueltos.

Un **mapa** es un objeto que asigna claves a valores o es una colección de pares atributo-valor. En la imagen siguiente se muestra un map:



Note que un **mapa** no se considera una colección verdadera, ya que la interfaz de mapa no extiende la interfaz de colección. En su lugar, inicia una rama independiente en Java Collections Framework, como se muestra en el diagrama siguiente:

Dado que **map** no es una colección verdadera, sus características y comportamientos son diferentes de las otras colecciones como **List** o **Set**. Un mapa no puede contener claves duplicadas y cada clave puede asignarse como máximo a un valor. Algunas implementaciones permiten la clave null y el valor null (**HashMap** y **LinkedHashMap**) pero algunas no lo hacen (**TreeMap**).

El orden de un mapa depende de implementaciones específicas, por ejemplo **TreeMap** y **LinkedHashMap** tienen un orden predecible, mientras **que HashMap** no.

3.4.1.1 *Por qué y cuándo usar mapas:*

Los mapas son perfectos para la asignación de asociaciones clave-valor, como diccionarios. Utilice Mapas cuando desee recuperar y actualizar elementos por claves o realizar búsquedas por claves. Algunos ejemplos:

- Un mapa de códigos de error y sus descripciones.
 - Un mapa de códigos postales y ciudades.
- Un mapa de gerentes y empleados. Cada gerente (clave) está asociado con una lista de empleados (valor) que administra.
 - Un mapa de clases y estudiantes. Cada clase (clave) está asociada a una lista de alumnos (valor).

Este tutorial proporciona ejemplos de código alrededor de las tres implementaciones principales de Map que se describen a continuación.

3.4.1.2 *Implementaciones del mapa*

En el árbol de herencia de la interfaz map, hay varias implementaciones, pero sólo vamos a ver 3 implementaciones principales, comunes y de propósito general que son **HashMap** y **LinkedHashMap** y **TreeMap**.

- **HashMap**: esta implementación utiliza una tabla hash como estructura de datos subyacente. Implementa todas las operaciones de mapa y permite valores nulos y una clave nula. Esta clase es aproximadamente equivalente a **Hashtable**: una estructura de datos heredada antes de Java Collections Framework, pero no está sincronizada y permite valores NULL. **HashMap** no garantiza el orden de sus elementos clave-valor. Por lo tanto, considere la posibilidad de usar un **HashMap** cuando el orden no importa y los valores NULL son aceptables.
- **LinkedHashMap**: esta implementación utiliza una tabla hash y una lista vinculada como estructuras de datos subyacentes, por lo que el orden de un **LinkedHashMap** es predecible, con el orden de inserción como orden predeterminado. Esta implementación también permite valores NULL como **HashMap**. Por lo tanto, considere la posibilidad de usar un **LinkedHashMap** cuando desee que un mapa con sus pares clave-valor se ordene por su orden de inserción.
- **TreeMap**: esta implementación utiliza un árbol rojo-negro como estructura de datos subyacente. Un **TreeMap** se ordena según el orden natural de sus claves, o por un **comparador** proporcionado en tiempo de creación. Esta implementación no permite valores NULL. Por lo tanto, considere la posibilidad de utilizar un **TreeMap** cuando desee que un mapa ordene sus pares clave-valor por el orden natural de las claves (por ejemplo, orden alfabético o orden numérico), o por un orden personalizado que especifique.

3.4.1.3 *Creación de un nuevo mapa*

3.4.1.3.1 Creación de un HashMap:

Utilice siempre el tipo de interfaz (**Mapa**), genéricos y operador de diamante para declarar un nuevo mapa. El código siguiente crea un **HashMap**:

```
1Map<Integer, String> mapHttpErrors = new HashMap<>();
2
3mapHttpErrors.put(200, "OK");
4mapHttpErrors.put(303, "See Other");
5mapHttpErrors.put(404, "Not Found");
6mapHttpErrors.put(500, "Internal Server Error");
7
8System.out.println(mapHttpErrors);
```

Esto asigna códigos de estado HTTP a sus descripciones. salida:

```
1{404=Not Found, 500=Internal Server Error, 200=OK, 303=See Other}
```

Como puede ver en la salida, un **HashMap** no impone ningún orden a sus elementos clave-valor. Puede crear un nuevo mapa que copie elementos de un mapa existente. por ejemplo:

```
1Map<Integer, String> mapErrors = new HashMap<>(mapHttpErrors);
```

El **mapa mapErrors** se crea con elementos iniciales copiados del **mapa mapHttpErrors**.

3.4.1.3.2 Creación de un LinkedHashMap:

El código siguiente crea un **LinkedHashMap** que asigna números de teléfono con nombres de contacto:

```
1Map<String, String> mapContacts = new LinkedHashMap<>();
2
3mapContacts.put("0169238175", "Tom");
4mapContacts.put("0904891321", "Peter");
5mapContacts.put("0945678912", "Mary");
6mapContacts.put("0981127421", "John");
7
8System.out.println(mapContacts);
```

salida:

```
1{0169238175=Tom, 0904891321=Peter, 0945678912=Mary, 0981127421=John}
```

Como puede ver, **LinkedHashMap** mantiene sus elementos por su orden de inserción.

3.4.1.3.3 Creación de un TreeMap:

El código siguiente crea un **TreeMap** que asigna extensiones de archivo a lenguajes de programación:

```

1Map<String, String> mapLang = new TreeMap<>();
2
3mapLang.put(".c", "C");
4mapLang.put(".java", "Java");
5mapLang.put(".pl", "Perl");
6mapLang.put(".cs", "C#");
7mapLang.put(".php", "PHP");
8mapLang.put(".cpp", "C++");
9mapLang.put(".xml", "XML");
10
11System.out.println(mapLang);

```

salida:

```
1{.c=C, .cpp=C++, .cs=C#, .java=Java, .php=PHP, .pl=Perl, .xml=XML}
```

Como puede ver, **treemap** ordena sus claves por su [orden natural](#), que es el orden alfabético en este caso.

3.4.1.4 Realización de operaciones básicas en un mapa

Las operaciones básicas de un mapa son asociación (**put**), búsqueda (**get**), comprobación (**containsKey** y **containsValue**), modificación (**quitar** y **reemplazar**) y cardinalidad (**tamaño** y **isEmpty**).

3.4.1.4.1 Asociar un valor con una clave:

El método **put(K, V)** asocia el valor especificado V con la clave especificada K. Si el mapa ya contiene una asignación para la clave, el valor antiguo se reemplaza por el valor especificado:

```

1Map<Integer, String> mapHttpErrors = new HashMap<>();
2mapHttpErrors.put(400, "Bad Request");
3mapHttpErrors.put(304, "Not Modified");
4mapHttpErrors.put(200, "OK");
5mapHttpErrors.put(301, "Moved Permanently");
6mapHttpErrors.put(500, "Internal Server Error");

```

3.4.1.4.2 Obtención de un valor asociado a una clave especificada:

El método **get(Object key)** devuelve el valor asociado a la clave especificada o devuelve null si el mapa no contiene ninguna asignación para la clave. Dado el mapa en el ejemplo anterior:

```

1String status301 = mapHttpErrors.get(301);
2System.out.println("301: " + status301);

```

salida:

```
1301: Moved Permanently
```

3.4.1.4.3 Comprobación de si el mapa contiene una clave especificada:

El método `containsKey(Object key)` devuelve true si el mapa contiene una asignación para la clave especificada. por ejemplo:

```
1if (mapHttpErrors.containsKey("200")) {  
2    System.out.println("Http status 200");  
3}
```

salida:

```
1Found: Http status 200
```

3.4.1.4.4 Comprobación de si el mapa contiene un valor especificado:

El método `containsValue(Object value)` devuelve true si el mapa contiene una o varias claves asociadas al valor especificado. por ejemplo:

```
1if (mapHttpErrors.containsValue("OK")) {  
2    System.out.println("Found status OK");  
3}
```

salida:

```
1Found status OK
```

3.4.1.4.5 Eliminación de una asignación del mapa:

El método `remove(Object key)` elimina la asignación de una clave del mapa si está presente (solo nos importa la clave y el valor no importa). Este método devuelve el valor al que el mapa asoció previamente la clave, o null si el mapa no contiene asignación para la clave. Este es un ejemplo:

```
1String removedValue = mapHttpErrors.remove(500);  
2  
3if (removedValue != null) {  
4    System.out.println("Removed value: " + removedValue);  
5}
```

salida:

```
1Removed value: Internal Server Error
```

De forma similar, el método `remove(Object key, Object value)` quita la asignación de una clave especificada y un valor especificado y devuelve true si se ha quitado el valor.

Este método es útil en caso de que realmente nos preocupamos por la clave y el valor a eliminar. Le recomiendo que lea este [conocido libro de colecciones Java](#) para aprender en profundidad sobre el marco de java collections.

3.4.1.4.6 Sustitución de un valor asociado a una clave especificada:

El método `replace(K key, V value)` reemplaza la entrada de la clave especificada solo si actualmente se está asignando a algún valor. Este método devuelve el valor anterior asociado a la clave especificada. Este es un ejemplo:

```
1System.out.println("Map before: " + mapHttpErrors);
2
3mapHttpErrors.replace(304, "No Changes");
4
5System.out.println("Map after: " + mapHttpErrors);
```

salida:

```
Map before: {400=Bad Request, 304=Not Modified, 200=OK, 301=Moved
1Permanently}
2Map after: {400=Bad Request, 304=No Changes, 200=OK, 301=Moved
Permanently}
```

De forma similar, el método `replace(K key, V oldValue, V newValue)` reemplaza la entrada de la clave especificada solo si se está asignando actualmente al valor especificado. Este método devuelve true si se ha reemplazado el valor. Útil en caso de que queramos reemplazar exactamente una asignación clave-valor.

3.4.1.4.7 Obtención del tamaño del mapa:

El método `size()` devuelve el número de asignaciones clave-valor en este mapa. por ejemplo:

```
1int size = mapHttpErrors.size();
```

salida:

```
1Number of HTTP status code: 5
```

3.4.1.4.8 Comprobación de si el mapa está vacío:

El método `isEmpty()` devuelve true si el mapa no contiene asignaciones clave-valor. por ejemplo:

```
1if (mapHttpErrors.isEmpty()) {
2    System.out.println("No Error");
3} else {
4    System.out.println("Have HTTP Errors");
5}
```

salida:

```
1Have HTTP Errors
```

3.4.1.5 Iteración sobre un mapa (utilizando vistas de colección)

Como un mapa no es una colección verdadera, no hay ningún método directo para iterar sobre un mapa. En su lugar, podemos iterar sobre un mapa utilizando sus vistas de colección. La implementación de cualquier mapa tiene que proporcionar los siguientes tres métodos de vista de colección:

- **keySet()**: devuelve una vista Set de las claves contenidas en el mapa. Por lo tanto, podemos iterar sobre las claves del mapa como se muestra en el siguiente ejemplo:

```
1Map<String, String> mapCountryCodes = new HashMap<>();
2
3mapCountryCodes.put("1", "USA");
4mapCountryCodes.put("44", "United Kingdom");
5mapCountryCodes.put("33", "France");
6mapCountryCodes.put("81", "Japan");
7
8Set<String> setCodes = mapCountryCodes.keySet();
9Iterator<String> iterator = setCodes.iterator();
10
11while (iterator.hasNext()) {
12    String code = iterator.next();
13    String country = mapCountryCodes.get(code);
14
15    System.out.println(code + " => " + country);
16}
```

salida:

```
144 => United Kingdom
233 => France
31 => USA
481 => Japan
```

- **values()**: devuelve una colección de valores contenidos en el mapa. Así podemos iterar sobre valores del mapa como este:

```
1Collection<String> countries = mapCountryCodes.values();
2
3for (String country : countries) {
4    System.out.println(country);
5}
```

salida:

```
1United Kingdom
2France
3USA
4Japan
```

- **entrySet()** : devuelve una vista Set de las asignaciones contenidas en este mapa. Por lo tanto, podemos iterar sobre mapeos en el mapa de esta manera:

```
1Set<Map.Entry<String, String>> entries = mapCountryCodes.entrySet();
2
3for (Map.Entry<String, String> entry : entries) {
4    String code = entry.getKey();
5    String country = entry.getValue();
6
7    System.out.println(code + " => " + country);
8}
```

salida:

```
144 => United Kingdom
233 => France
31 => USA
481 => Japan
```

Desde Java 8 con expresiones lambda y la instrucción **forEach()**, iterar sobre un mapa es tan fácil como:

```
1mapCountryCodes.forEach(
2    (code, country) -> System.out.println(code + " => " + country));
```

salida:

```
144 => United Kingdom
233 => France
31 => USA
481 => Japan
```

Para obtener más información acerca de los diferentes métodos de iteración de recopilación, lea este artículo: [Los 4 métodos para iterar colecciones en Java](#).

3.4.1.6 Realización de operaciones masivas con mapas

Hay dos operaciones a granel con mapas: **clear()** y **putAll()**. El método **clear()** elimina todas las asignaciones del mapa. El mapa estará vacío después de que este método vuelva. por ejemplo:

```
1mapHttpErrors.clear();
2System.out.println("Is map empty? " + mapHttpErrors.isEmpty());
```

salida:

1Is map empty? true

El método `putAll(Map<K, V> m)` copia todas las asignaciones del mapa especificado en este mapa. Este es un ejemplo:

```
1Map<Integer, String> countryCodesEU = new HashMap<>();
2
3countryCodesEU.put(44, "United Kingdom");
4countryCodesEU.put(33, "France");
5countryCodesEU.put(49, "Germany");
6
7Map<Integer, String> countryCodesWorld = new HashMap<>();
8
9countryCodesWorld.put(1, "United States");
10countryCodesWorld.put(86, "China");
11countryCodesWorld.put(82, "South Korea");
12
13
14System.out.println("Before: " + countryCodesWorld);
15
16countryCodesWorld.putAll(countryCodesEU);
17
18System.out.println("After: " + countryCodesWorld);
```

salida:

```
1Before: {1=United States, 82=South Korea, 86=China}
2After: {1=United States, 33=France, 49=Germany, 82=South Korea, 86=China,
44=United Kingdom}
```

3.4.1.7 Mapas simultáneos

A diferencia de la `tabla hash heredada` que se sincroniza, `HashMap`, `TreeMap` y `LinkedHashMap` no se sincronizan. Si thread-safe es prioritario, considere la posibilidad de usar `ConcurrentHashMap` en lugar de `HashMap`. O podemos usar el método de utilidad `Collections.synchronizedMap()` que devuelve un mapa sincronizado (seguro para subprocesos) respaldado por el mapa especificado. por ejemplo:

```
1Map<Integer, String> map = Collections.synchronizedMap(new HashMap<>());
```

Y recuerde que tenemos que sincronizar manualmente el mapa al iterar sobre cualquiera de sus vistas de colección:

```
1Set<Integer> keySet = map.keySet();
2
3synchronized (map) {
4    Iterator<Integer> iterator = keySet.iterator();
5
6    while (iterator.hasNext()) {
7        Integer key = iterator.next();
8        String value = map.get(key);
9    }
10}
```

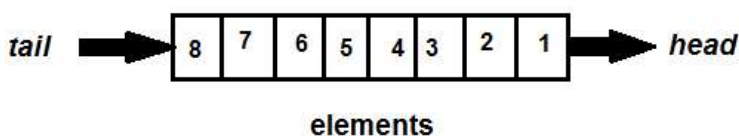
Si utiliza un tipo de `SortedMap`, por ejemplo `TreeMap`, considere la posibilidad de utilizar el método más específico `Collections.synchronizedSortedMap()`. **NOTA:** Si utiliza su propio tipo para la clave y el valor (por ejemplo, `Student` o `Employee`), la clase de clave y la clase de valor deben implementar correctamente los métodos `equals()` y `hashCode()` para que el mapa pueda buscarlos correctamente.

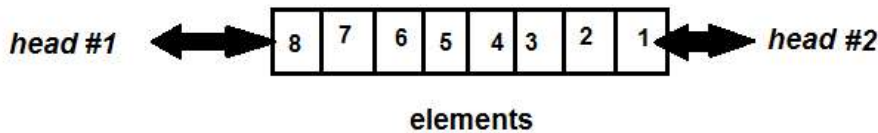
3.5 Cola

Cola significa "línea de espera", que es muy similar a las colas en la vida real: una cola de personas de pie en la puerta de facturación de un aeropuerto; una cola de coches esperando luz verde en una carretera de la ciudad; una cola de clientes esperando ser atendidos en el mostrador de un banco, etc. En la programación, la cola es una estructura de datos que contiene elementos antes del procesamiento, similar a las colas en escenarios de la vida real. Consideremos que una cola contiene una lista de clientes en espera en el mostrador de un banco. Cada cliente es atendido uno tras otro, siga el pedido en el que aparece o se registra. El primer cliente viene se sirve primero, y después de él es el segundo y el tercero y así sucesivamente. Cuando se realiza la administración de un cliente, deja el contador (eliminado de la cola) y el siguiente cliente se elige para ser atendido a continuación. Otros clientes vienen más tarde se agregan al final de la cola. Este procesamiento se denomina First In First Out o FIFO. Durante el procesamiento, la cola se puede cambiar dinámicamente, es decir, los elementos procesados se quitan de la cola y se agregan nuevos elementos a la cola. En Java Collections Framework, `Queue` es la interfaz principal y hay cuatro sub interfaces: `Deque`, `BlockingDeque`, `BlockingQueue` y `TransferQueue`. Excepto la interfaz `Deque` que se encuentra en el paquete `java.util`, todos los demás se organizan en el paquete `java.util.concurrent`, que está diseñado para la programación [multiproceso](#) o simultánea.

3.5.1.1 Características de la cola

Básicamente, una cola tiene cabeza y cola. Se agregan nuevos elementos a la cola y los elementos que se van a procesar se seleccionan de la cabeza. La siguiente imagen ilustra una cola típica: los elementos de la cola se mantienen mediante su orden de inserción. La interfaz `Queue` abstrae este tipo de cola. Otro tipo de cola es cola de doble extremo o deque. Un deque tiene dos cabezas, lo que permite agregar o quitar elementos de ambos extremos. La siguiente imagen ilustra este tipo de cola: la interfaz `Deque` abstrae este tipo de cola y es una subinterfaz de la interfaz `Queue`. Y la clase `LinkedList` es una implementación conocida. Algunas implementaciones aceptan elementos null, otras no. `Queue` permite elementos duplicados, porque la característica principal de la cola mantiene los elementos por su orden de inserción. Los elementos duplicados en términos de contrato igual se consideran distintos en términos de cola, ya que no hay dos elementos que tengan el mismo orden. Además, Java Collection Framework proporciona la interfaz `BlockingQueue` que abstrae las colas que se pueden utilizar en contexto simultáneo (multiproceso). Una cola de bloqueo espera a que la cola no esté vacía al recuperar un elemento y espera a que el espacio esté disponible en la cola al almacenar un elemento. Del mismo modo, la interfaz `BlockingDeque` está bloqueando la cola para colas de doble extremo.





3.5.1.2 Comportamientos de la cola

Debido a la naturaleza de `Queue`, las operaciones clave que diferencian `Queue` de otras colecciones son la extracción y la inspección en la cabecera de la cola. Para los deque, la extracción y la inspección se pueden procesar en ambos extremos. Y debido a que la interfaz `cola` extiende la interfaz `de colección`, todas las implementaciones de cola proporcionan operaciones principales de una colección como `add()`, `contains()`, `remove()`, `clear()`, `isEmpty()`, etc. Y tenga en cuenta que, con las colas, las operaciones en la cabeza son más rápidas (por ejemplo, `offer()` y `remove()`), mientras que las operaciones en elementos intermedios son lentas (por ejemplo, `contains(e)` y `remove(e)`).

3.5.1.3 Interfaces de cola

La `cola` es la superinterfaz de la rama de cola en Java Collection Framework. Debajo de él, hay las siguientes sub interfaces:

- **Deque**: abstrae una cola que tiene dos cabezas. Un deque permite agregar o eliminar elementos en ambos extremos.
 - **BlockingQueue**: abstrae un tipo de colas que espera a que la cola no esté vacía al recuperar un elemento y espera a que el espacio esté disponible en la cola al almacenar un elemento.
 - **BlockingDeque**: es similar a `BlockingQueue`, pero para colas de doble extremo. Es la subinterfaz del `BlockingQueue`.

Y desde Java 7, la interfaz `BlockingQueue` tiene una nueva subinterfaz llamada `TransferQueue`, que es un `BlockingQueue` especializado, que espera a que otro subproceso recupere un elemento en la cola.

3.5.1.4 Implementaciones de la cola principal

El marco de Java Collection proporciona muchas implementaciones, principalmente para la interfaz `BlockingQueue`. A continuación noto pocos que se utilizan comúnmente. Las implementaciones de cola se agrupan en dos grupos: implementaciones simultáneas y de uso general.

3.5.1.4.1 Implementaciones de cola de uso general:

- + **LinkedList**: esta clase implementa la interfaz `List` y `Deque`, con características híbridas y comportamientos de lista y cola. Considere la posibilidad de usar un `LinkedList` cuando desee agregar rápidamente y quitar elementos rápidos en ambos extremos, además de acceder a los elementos por índice.
- + **PriorityQueue**: esta cola ordena elementos según su orden natural, o por un `comparador` proporcionado en tiempo de construcción. Considere la posibilidad de utilizar un `PriorityQueue` cuando desee aprovechar el orden natural y agregar rápidamente elementos a la cola y eliminar rápidamente elementos en la cabeza de la cola.

+ **ArrayDeque**: una implementación sencilla de la interfaz **Deque**. Considere la posibilidad de usar un **ArrayDeque** cuando desee utilizar características de una cola de doble extremo sin las basadas en listas (más simple que un **LinkedList**).

3.5.1.4.2 Implementaciones simultáneas de colas:

+ **ArrayBlockingQueue**: se trata de una cola de bloqueo respaldada por una matriz. Considere la posibilidad de usar un **ArrayBlockingQueue** cuando desee utilizar una cola de bloqueo simple que tenga capacidad limitada (delimitada).

+ **PriorityBlockingQueue**: Utilice esta clase cuando desee aprovechar **PriorityQueue** y **BlockingQueue**.

+ **DelayQueue**: una cola de bloqueo de programación basada en el tiempo. Los elementos agregados a esta cola deben implementar la interfaz **retrasada**. Esto significa que un elemento solo se puede tomar de la cabeza de la cola cuando su retraso ha expirado.

Dado que la interfaz **de cola** amplía la interfaz **de colección**, todas las implementaciones **de cola** tienen operaciones básicas de una colección:

- Operaciones individuales: `add(e)`, `contains(e)`, `iterator()`, `clear()`, `isEmpty()`, `size()` y `toArray()`.
- Operaciones masivas: `addAll()`, `containsAll()`, `removeAll()` y `retainAll()`.

3.5.1.5 Comprender la estructura de api de cola

3.5.1.5.1 Comprender la estructura api de la interfaz de cola

Básicamente, **Queue** proporciona tres tipos principales de operaciones que diferencian una cola de otras:

- Insertar: agrega un elemento a la cola de la cola.
- Quitar: quita el elemento en la cabeza de la cola.
- Examinar: devuelve, pero no quita, el elemento en la cabeza de la cola.

Y para cada tipo de operación, hay dos versiones:

- La primera versión produce una excepción si se produce un error en la operación, por ejemplo, no se pudo agregar elemento cuando la cola está llena.
- La segunda versión devuelve un valor especial (null o false, dependiendo de la operación).

En la tabla siguiente se resumen las operaciones principales de la interfaz **cola**:

<i>Tipo de operación</i>	<i>Lanza excepción</i>	<i>Devuelve un valor especial</i>
insertar	<code>add(e)</code>	<code>oferta(e)</code>
eliminar	<code>remove()</code>	<code>encuesta()</code>
examinar	<code>elemento()</code>	<code>peek()</code>

3.5.1.5.2 Comprensión de la estructura api de la interfaz de Deque

Como saben, la interfaz **Deque** abstrae una cola de doble extremo con dos extremos (primero y último), por lo que su API se estructura en torno a esta característica. Una

implementación de `Deque` proporciona los métodos `xxxFirst()` que funcionan en el primer elemento y los métodos `xxxLast()` que funcionan en el último elemento. En la tabla siguiente se resume la estructura api de `Deque`:

<i>Tipo de operación</i>	<i>Primer elemento</i>	<i>Último elemento</i>
insertar	<code>addFirst(e)</code> <code>ofertaPrim(e)</code>	<code>addLast(e)</code> <code>ofertaLast(e)</code>
eliminar	<code>removeFirst()</code> <code>pollFirst()</code>	<code>removeLast()</code> <code>pollLast()</code>
examinar	<code>getFirst()</code> <code>peekFirst()</code>	<code>getLast()</code> <code>peekLast()</code>

3.5.1.5.3 Comprensión de la estructura api de la interfaz `BlockingQueue`

Una cola de bloqueo está diseñada para esperar a que la cola no esté vacía al recuperar un elemento (el método `put(e)` y esperar a que el espacio esté disponible en la cola al almacenar un método `take()` de elemento. Además, una cola de bloqueo proporciona operaciones especializadas que pueden esperar hasta una duración especificada al insertar y quitar un elemento. En la tabla siguiente se resume la estructura api de la interfaz `BlockingQueue`:

<i>Tipo de operación</i>	<i>Lanza excepción</i>	<i>valor especial</i>	<i>Bloques</i>	<i>tiempos fuera</i>
insertar	<code>add(e)</code>	<code>oferta(e)</code>	<code>put(e)</code>	<code>oferta(e, tiempo, unidad)</code>
eliminar	<code>remove()</code>	<code>encuesta()</code>	<code>take()</code>	<code>encuesta (tiempo, unidad)</code>
examinar	<code>elemento()</code>	<code>peek()</code>	N/A	N/A

3.5.1.5.4 Comprensión de la estructura api de la interfaz `BlockingDeque`

Del mismo modo, un `BlockingDeque` es un `BlockingQueue` especializado para cola de doble extremo con dos extremos (cabeza y cola). Su API está en esquema de métodos `xxxFirst()` que operan en los métodos first element y `xxxLast()` que operan en el último elemento. En la tabla siguiente se resume la estructura api de `BlockingDeque`:

<i>Primer elemento (cabeza)</i>				
	<i>Lanza excepción</i>	<i>valor especial</i>	<i>Bloques</i>	<i>tiempos fuera</i>
insertar	<code>addFirst(e)</code>	<code>ofertaPrim(e)</code>	<code>putFirst(e)</code>	<code>ofertaPrim(e, tiempo, unidad)</code>
eliminar	<code>removeFirst()</code>	<code>pollFirst()</code>	<code>takeFirst()</code>	<code>pollFirst(tiempo, unidad)</code>
examinar	<code>getFirst()</code>	<code>peekFirst()</code>	N/A	N/A
<i>Último elemento (cola)</i>				
	<i>Lanza excepción</i>	<i>valor especial</i>	<i>Bloques</i>	<i>tiempos fuera</i>
insertar	<code>addLast(e)</code>	<code>ofertaLast(e)</code>	<code>putLast(e)</code>	<code>ofertaLast(e, tiempo, unidad)</code>
eliminar	<code>removeLast()</code>	<code>pollLast()</code>	<code>takeLast()</code>	<code>pollLast(tiempo, unidad)</code>
examinar	<code>getLast()</code>	<code>peekLast()</code>	N/A	N/A

Repasemos varios ejemplos de código para comprender cómo usar colecciones `queue` en la codificación diaria. En los próximos ejemplos, utilizo implementaciones diferentes como `LinkedList`, `ArrayDeque`, `PriorityQueue`, `ArrayBlockingQueue`, ec. Para obtener información detallada sobre java collections framework, le recomiendo que lea el conocido libro [Java Generics and Collections](#).

3.5.1.6 Realización de operaciones en colecciones de colas

3.5.1.6.1 Creación de una nueva instancia de cola

Como práctica recomendada, se recomienda usar el tipo genérico y la interfaz como tipo de referencia al crear una nueva colección. Para las colas, dependiendo de la necesidad de un tipo determinado (cola, cola y cola de bloqueo), utilice la interfaz correspondiente como tipo de referencia. Por ejemplo, las siguientes instrucciones crean 3 tipos diferentes de colas:

```
1Queue<String> namesQueue = new LinkedList<>();
2Deque<Integer> numbersDeque = new ArrayDeque<>();
3BlockingQueue<String> waitingCustomers = new ArrayBlockingQueue<>(100);
```

La mayoría de las implementaciones de cola no tienen restricción en la capacidad (colas sin enlazar), excepto las clases `ArrayBlockingQueue`, `LinkedBlockingQueue` y `LinkedBlockingDeque`. La siguiente instrucción crea un `ArrayBlockingQueue` con capacidad fija de 200 elementos:

```
1BlockingQueue<String> waitingCustomers = new ArrayBlockingQueue<>(200);
```

Recuerde también que podemos usar el constructor de copia para crear una nueva instancia de `Queue` a partir de otra colección. por ejemplo:

```
1List<String> listNames = Arrays.asList("Alice", "Bob", "Cole", "Dale",
2"Eric", "Frank");
3Queue<String> queueNames = new LinkedList<>(listNames);
4System.out.println(queueNames);
```

salida:

```
1[Alice, Bob, Cole, Dale, Eric, Frank]
```

3.5.1.6.2 Adición de nuevos elementos a la cola

Para insertar un elemento en la cola de la cola, podemos utilizar el método `add()` u `offer()`. El código siguiente agrega dos elementos a una lista vinculada:

```
1Queue<String> queueNames = new LinkedList<>();
2queueNames.add("Mary");
3queueNames.add("John");
```

Cuando se utiliza una cola sin enlazar (sin restricción de capacidad), los métodos `add()` y `offer()` no muestran la diferencia. Sin embargo, cuando se utiliza una cola delimitada, el

método `add()` producirá una excepción si la cola está llena, mientras que el método `offer()` devuelve `false`. En el ejemplo siguiente se muestra esta diferencia:

```
1Queue<Integer> queueNumbers = new ArrayBlockingQueue<>(3);
2queueNumbers.add(1);
3queueNumbers.add(2);
4queueNumbers.add(3);
5queueNumbers.add(4);    // this line throws exception
```

La última línea produce `java.lang.IllegalStateException: Cola completa` porque declaramos la cola con capacidad de 3 elementos. Por lo tanto, la adición de la 4^{ésimo} elemento da como resultado una excepción. Sin embargo, estamos seguros cuando utilizamos el método `offer()`, como se muestra en el siguiente fragmento de código:

```
1Queue<Integer> queueNumbers = new ArrayBlockingQueue<>(3);
2
3System.out.println(queueNumbers.offer(1));
4System.out.println(queueNumbers.offer(2));
5System.out.println(queueNumbers.offer(3));
6System.out.println(queueNumbers.offer(4));
```

salida:

```
1true
2true
3true
4false
```

El siguiente fragmento de código agrega un elemento a la cabeza y un elemento a la cola de una cola de doble extremo (observe el tipo de interfaz que se utiliza):

```
1Deque<String> queueNames = new ArrayDeque<>();
2
3queueNames.offer("Katherine");
4queueNames.offer("Bob");
5
6queueNames.addFirst("Jim");
7queueNames.addLast("Tom");
8
9System.out.println(queueNames);
```

salida:

```
1[Jim, Katherine, Bob, Tom]
```

Para bloquear la cola, utilice `put(e)` u `offer(e, time, unit)` en caso de que desee que el subproceso actual espere hasta que el espacio esté disponible en la cola. por ejemplo:

```

1BlockingQueue<Integer> queueNumbers = new ArrayBlockingQueue<>(100);
2
3try {
4    queueNumbers.put(2000);
5} catch (InterruptedException ie) {
6    ie.printStackTrace();
7}

```

3.5.1.6.3 Extracción de la cabeza de la cola

Una cola proporciona los métodos `remove()` y `poll()` que nos permiten seleccionar el elemento en la cabeza y eliminarlo de la cola. Y usted debe entender la diferencia entre estos dos métodos. El método `remove()` devuelve el elemento head o produce una excepción si la cola está vacía. por ejemplo:

```

1Queue<String> queueCustomers = new LinkedList<>();
2queueCustomers.offer("Jack");
3
4String next = queueCustomers.remove();
5System.out.println("Next customer is: "+ next);
6
7next = queueCustomers.remove(); // throws exception

```

Aquí, la cola solo tiene un elemento, por lo que la primera llamada a `remove()` funciona bien. Sin embargo, la invocación posterior da como resultado `java.util.NoSuchElementException` porque la cola se vacía. Por el contrario, el método `poll()` devuelve `null` si la cola está vacía, como se muestra en el ejemplo siguiente:

```

1Queue<String> queueCustomers = new LinkedList<>();
2queueCustomers.offer("Jack");
3
4System.out.println("next: " + queueCustomers.poll());
5System.out.println("next: " + queueCustomers.poll()); // returns null

```

salida:

```

1next: Jack
2next: null

```

En el ejemplo siguiente se elimina el elemento de cabeza y el elemento de cola de un deque:

```

1Deque<String> queueCustomers = new ArrayDeque<>();
2
3queueCustomers.offer("Bill");
4queueCustomers.offer("Kim");
5queueCustomers.offer("Lee");
6queueCustomers.offer("Peter");
7queueCustomers.offer("Sam");
8
9System.out.println("Queue before: " + queueCustomers);
10System.out.println("First comes: " + queueCustomers.pollFirst());
11System.out.println("Last comes: " + queueCustomers.pollLast());
12System.out.println("Queue after: " + queueCustomers);

```

salida:

```

1Queue before: [Bill, Kim, Lee, Peter, Sam]
2First comes: Bill
3Last comes: Sam
4Queue after: [Kim, Lee, Peter]

```

Para una cola de bloqueo, utilice los métodos `take()` o `poll(time, unit)` en caso de que desee que el subproceso actual espere hasta que un elemento esté disponible. por ejemplo:

```

1BlockingQueue<String> queueCustomers = new ArrayBlockingQueue<>(100);
2
3queueCustomers.offer("Kathe");
4
5try {
6    String nextCustomer = queueCustomers.take();
7} catch (InterruptedException ie) {
8    ie.printStackTrace();
9}

```

3.5.1.6.4 Examinar la cabeza de la cola

A diferencia del método `remove()`, los métodos examinen `element()` y `peek()` devuelven (pero no quitan) la cabeza de la cola. Por lo tanto, considere la posibilidad de utilizar estos métodos en caso de que solo desee comprobar lo que está actualmente en el elemento principal sin modificar la cola. Además, debe conocer la diferencia entre los métodos `element()` y `peek()`: el método `element()` produce una excepción en caso de que la cola esté vacía, mientras que el método `peek()` devuelve `null`. por ejemplo:

```

1Queue<String> queueCustomers = new PriorityQueue<>();
2
3queueCustomers.offer("Jack");
4
5System.out.println("who's next: " + queueCustomers.poll());
6
7// this returns null in case the queue is empty
8System.out.println("who's next: " + queueCustomers.peek());
9
10// this throws exception in case the queue is empty
11System.out.println("who's next: " + queueCustomers.element());

```

Para un deque, utilice los métodos `getFirst()` o `peekFirst()` para examinar el primer elemento y `getLast()` o `peekLast()` para examinar el último elemento. Este es un ejemplo:

```

1Deque<Integer> queueNumbers = new ArrayDeque<>();
2
3queueNumbers.add(10);
4queueNumbers.add(20);
5queueNumbers.add(30);
6queueNumbers.add(40);
7
8System.out.println("first: " + queueNumbers.getFirst());
9System.out.println("last: " + queueNumbers.peekLast());

```

No hay ningún método para examinar una cola de bloqueo.

3.5.1.7 Iteración sobre elementos en la cola

Podemos utilizar el método mejorado para bucle, iterador y `forEach()` para recorrer en iteración los elementos de la cola. El siguiente fragmento de código ilustra cómo recorrer en iteración una lista vinculada mediante el bucle `for` mejorado:

```

1Queue<String> queueNames = new LinkedList<>();
2
3queueNames.add("Dale");
4queueNames.add("Bob");
5queueNames.add("Frank");
6queueNames.add("Alice");
7queueNames.add("Eric");
8queueNames.add("Cole");
9queueNames.add("John");
10
11
12for (String name : queueNames) {
13    System.out.println(name);
14}

```

salida:

```
1Dale
2Bob
3Frank
4Alice
5Eric
6Cole
7John
```

Más sencillamente, el uso de la expresión Lambda con el método `forEach()` en Java 8:

```
1queueNames.forEach(name -> System.out.println(name));
```

En el ejemplo siguiente se recorren en iteración los elementos de un `PriorityQueue` que ordena los elementos por orden natural:

```
1Queue<String> queueNames = new PriorityQueue<>();
2
3queueNames.add("Dale");
4queueNames.add("Bob");
5queueNames.add("Frank");
6queueNames.add("Alice");
7queueNames.add("Eric");
8queueNames.add("Cole");
9queueNames.add("John");
10
11queueNames.forEach(name -> System.out.println(name));
```

salida:

```
1Alice
2Bob
3Cole
4Dale
5Eric
6Frank
7John
```

Como se puede ver en la salida, los elementos se ordenan en el orden alfabético (orden natural de Cadenas). **NOTA:** Preste atención al utilizar un iterador de un `PriorityQueue`, porque no está garantizado atravesar los elementos de la cola de prioridad en ningún orden determinado.

3.5.1.8 Colas simultáneas

Todas las implementaciones de `BlockingQueue` son seguras para subprocesos. Las siguientes implementaciones no son:

- `LinkedList`
- `ArrayDeque`
- `PrioridadQueue`

Cuando desee utilizar una lista vinculada sincronizada, utilice el código siguiente:

```
1List list = Collections.synchronizedList(new LinkedList<>());
```

Y considere la posibilidad de usar `priorityblockingqueue` en lugar de `PriorityQueue` cuando desee utilizar una cola de prioridad sincronizada.

Bibliografía:

<https://www.codejava.net/java-core/collections/>

[Collections en Java \(reloco.com.ar\)](#)